

CORE DIRECTIVES

@for

```
@for (course of courses; track course.id) {  
  <course-card [course]="course"></course-card>  
}
```

Angular ha bisogno di una funzione di tracciamento (tracking function), altrimenti otterrai un errore di compilazione.

La **funzione di tracciamento** aiuta Angular ad aggiornare la lista usando un identificatore univoco; nella direttiva **ngFor**, la funzione di tracciamento esisteva già, ma non era obbligatoria.

Possiamo usare **@empty** se vogliamo mostrare qualcosa di diverso nel caso in cui l'array sia vuoto.

```
@for (course of courses; track course.id) {  
  <course-card [course]="course"></course-card>  
}  
@empty {  
  <h1>No element found</h1>  
}
```

Possiamo anche accedere all'indice dell'elemento usando **\$index** e al conteggio totale degli elementi usando **\$count**:

```
@for (course of courses; track course.id; let index = $index; let count = $count) {  
  <course-card [course]="course"></course-card>  
}  
@empty {  
  <h1>No element found</h1>  
}
```

Abbiamo anche altre variabili come **\$first** e **\$last**, che sono booleani utili, ad esempio, per applicare uno stile al primo e all'ultimo elemento dell'array.

Allo stesso modo possiamo usare, per scopi simili, **\$even** e **\$odd**.

Nota che tutte queste variabili possono essere usate direttamente senza bisogno di rinominarle, ad esempio:

```
@for (course of courses; track course.id; let first = $index; let count = $count) {  
  <course-card  
    [course]="course"  
    [class.is-even]="$even"  
    [class.is-odd]="$odd"  
  ></course-card>  
}  
@empty {  
  <h1>No element found</h1>  
}
```

ngFor

La direttiva ngFor ha invece la seguente sintassi:

```
<course-card *ngFor="let course of courses; index as i;  
  first as isFirst; last as isLast;  
  even as isEven; odd as isOdd"  
[course]="course" [class.is-even]="isEven"  
></course-card>
```

@If

```
@if(course.imageUrl) {  
  <img [src]="course.imageUrl" alt="">  
}  
@else if (course.id === 1){  
  <h1>course with id n 1</h1>  
}  
@else {  
  <h2>No image available</h2>  
}
```

ngIf

```
<img [src]="course.imageUrl" alt="" *ngIf="course.imageUrl; else noImage">  
<ng-template #noImage>  
  <h1>no image available</h1>  
</ng-template>
```

@switch

```
@switch(course.category){  
  @case ("BEGINNER"){  
    <h1>Beginner</h1>  
  }  
  @case ("INTERMEDIATE"){  
    <h1>Intermediate</h1>  
  }  
  @case ("ADVANCED"){  
    <h1>Advanced</h1>  
  }  
  @default {  
    <h1>Unknow</h1>  
  }  
}
```

ngSwitch

```
<ng-container [ngSwitch]="course.category">  
  <h1 *ngSwitchCase="'BEGINNER'">Beginner</h1>  
  <h1 *ngSwitchCase="'INTERMEDIATE'">Intermediate</h1>  
  <h1 *ngSwitchCase="'ADVANCED'">Advanced</h1>  
  <h1 *ngSwitchDefault>Unknow</h1>  
</ng-container>
```

ng-container

È utile quando non hai bisogno di creare un elemento DOM aggiuntivo solo per applicare una direttiva (non creerà alcun elemento nel DOM).

Attribute directives

Una direttiva è una classe che può modificare il comportamento o l'aspetto degli elementi nel DOM. Per ora abbiamo parlato di **direttive strutturali**, ma ne esistono molte altre, ad esempio:

```
<form>
  <input disabled required>
</form>
```

disabled e required sono **direttive attributo**.

Possiamo anche creare una nostra direttiva attributo personalizzata usando il comando **Angular CLI**:

```
ng g directive directive/highlighted
```

E generiamo:

```
import { Directive } from '@angular/core';

@Directive({
  selector: '[highlighted]'
})
export class HighlightedDirective {

  constructor() { }

}
```

Possiamo applicare la direttiva usando il suo **selettore**:

```
<course-card highlighted
  (courseSelected)="onCourseSelected($event)"
  [course]="course">

  <course-image [src]="course.iconUrl"></course-image>

  <div class="course-description">
    {{ course.longDescription }}
  </div>

</course-card>
```

Ora questa particolare istanza del componente **course-card** è conosciuta come l'**host element** della direttiva **highlighted**.

Una direttiva viene sempre applicata a un **elemento host**.

Supponiamo, ad esempio, di voler applicare la classe CSS **highlighted** all'elemento host della direttiva.

Possiamo interagire con l'elemento host usando il decoratore **@HostBinding**:

```
@HostBinding('className')
get cssClasses() {
  return "highlighted"
}
```

Il decoratore accetta come argomento la proprietà del DOM dell'host che vogliamo modificare e, per cambiarla, dobbiamo definire una funzione **get** che restituisce il valore che vogliamo assegnare.

Per questo esempio specifico (aggiungere una classe all'host), possiamo anche scrivere:

```
@HostBinding('class.highlighted')
get cssClasses() {
  return true
}
```

In modo simile possiamo anche modificare lo stile dell'elemento host:

```
@HostBinding('style.border')
get cssClasses() {
  return "1px solid red"
}
```

Possiamo anche avere una proprietà **Input** nella nostra direttiva.

Ad esempio, supponiamo di voler aggiungere la nostra classe all'elemento host in modo condizionale:

```
<course-card [highlighted]="true">

@Input('highlighted')
isHighlighted = false;

constructor() { }

@HostBinding('class.highlighted')
get cssClasses() {
  return this.isHighlighted
}
```

Possiamo anche modificare gli **attributi del DOM**:

```
@HostBinding('attr.disabled')
get disabled() {
  return "true"
}
```

Supponiamo di voler applicare la classe CSS **highlighted** ogni volta che passiamo il mouse sopra la card.

Possiamo farlo usando il decoratore **@HostListener**.

Il decoratore **HostListener** accetta come argomento una stringa che rappresenta l'evento DOM che vogliamo intercettare.

In questo caso, andremo a intercettare l'evento nativo **mouseover** sul nostro elemento host.

Ecco la logica che applicheremo ogni volta che passiamo il mouse sopra questo elemento:

```
@HostListener('mouseover')
mouseOver() {
  this.isHighlighted = true;
}
```

Possiamo accedere agli eventi nativi usando questa sintassi:

```
@HostListener('mouseover', ['$event'])
mouseover($event) {
  console.log($event)
  this.isHighlighted = true;
}
```

Anche le **direttive** possono emettere eventi:

```
@Output()
toggleHighlight = new EventEmitter();

@HostListener('mouseover', ['$event'])
mouseover($event) {
  console.log($event)
  this.isHighlighted = true;
  this.toggleHighlight.emit(true)
}

<course-card highlighted (toggleHighlight)="onToggle($event)"
```

Vediamo ora come possiamo implementare una **direttiva strutturale**:

```
@Directive({
  selector: '[ngxUnless]'
})
export class NgxUnlessDirective {

  constructor(private templateRef: TemplateRef<any>,
    private viewContainer: ViewContainerRef
  ) { }

  @Input()
  set ngxUnless(condition: boolean) {
    if(!condition){
      this.viewContainer.createEmbeddedView(this.templateRef)
    }else {
      this.viewContainer.clear()
    }
  }
}
```

Una delle caratteristiche chiave di una **direttiva strutturale**, rispetto a una semplice direttiva attributo, è la capacità di **istanzare** un template.

Quindi, la prima cosa che dobbiamo fare in questa direttiva è ottenere un riferimento a quel template. Possiamo farlo iniettando direttamente il **TemplateRef** nel costruttore.

Se dichiariamo una variabile che chiameremo templateRef, questa sarà un riferimento programmatico al nostro template.

Con questa variabile, saremo in grado di istanziare (ovvero far comparire) il template in un punto qualunque della direttiva.

Ricordiamo che il **template da solo non renderizza l'HTML**, serve un ulteriore passaggio per farlo apparire nel DOM.

Abbiamo bisogno di un modo per **istanzare** (rendere visibile) il template.

Per farlo, inietteremo un altro servizio integrato di Angular chiamato **ViewContainerRef**, che ci permette di gestire le **view** all'interno del DOM.

Infine, per applicare la nostra logica, ci servirà una proprietà **@Input** e una **funzione setter** per reagire ai cambiamenti del valore passato alla direttiva.